El algoritmo de Dijkstra

Grupo 2

18 de abril de 2026

UNIVERSIDAD DE CÓRDOBA

Integrantes del grupo

Alijo Fernández, Miguel Ángel
Buzón García, Alejandro
Casado Caballero, Antonio
Collantes Aguilar, Pablo
Deogracias Moya, Juan
Guerrero Romero, Jose Francisco
Laredo Gutiérrez, José Raúl
López Reyes, Marcos
Aramyan, David Ángel
Costa Ruiz-Canela, Javier Jesús
Ligero González, Alejandro

UNIVERSIDAD DE CÓRDOBA

Tabla de contenidos

1. Introducción
2. Descripción del algoritmo
3. Fundamento matemático
4. Ejemplo: Desarrollo paso a paso
5. Explicación del pseudocódigo
6. Simulación del algoritmo
7. Conclusiones
8. Bibliografía y webgrafía

UNIVERSIDAD DE CÓRDOBA

Introducción

Pregunta inicial

Si existen varios caminos para llegar de un punto a otro, ¿cómo podemos determinar cuál es el más eficiente?

- En muchos problemas no basta con llegar al destino.
- También interesa minimizar la distancia, el tiempo, el coste o los recursos.
- Este desafío se conoce como el **problema del camino mínimo**.

El algoritmo de Dijkstra permite abordar este problema en grafos ponderados con pesos no negativos.

UNIVERSIDAD DE CÓRDOBA

Motivación del problema

El problema del camino mínimo aparece en multitud de situaciones reales:

- **Sistemas GPS:** para calcular rutas eficientes.
- **Redes informáticas:** para enviar datos por los canales adecuados.
- **Logística y transporte:** para reducir costes y tiempos de entrega.
- **Planificación urbana:** para trazar recorridos en mapas o redes.

Por ello, su estudio es fundamental tanto en la Matemática Discreta como en la Informática.

UNIVERSIDAD DE CÓRDOBA

Objetivo del trabajo

En este trabajo analizaremos el algoritmo de Dijkstra como la técnica principal para resolver el problema del camino mínimo.

Comprender su funcionamiento nos permitirá dominar una de las herramientas más potentes y aplicadas de la teoría de grafos.

UNIVERSIDAD DE CÓRDOBA

Descripción del algoritmo

Fundamento matemático

Para comprender el algoritmo de Dijkstra desde una perspectiva rigurosa, es necesario establecer los fundamentos que garantizan su correctitud. Esta sección presenta las definiciones formales y las propiedades estructurales en las que se basa el algoritmo.

Definición (Grafo ponderado)

Un grafo ponderado es un par (G, w), donde $G = (V, E)$ es un grafo y $w : E \to \mathbb{R}^+$ es una función que asigna pesos positivos a las aristas del grafo.

UNIVERSIDAD DE CÓRDOBA

Fundamento matemático: Camino y Distancia

Definición (Camino y su peso)

Dado un grafo ponderado (G, w) y un camino $C : v_0 v_1 \ldots v_k$, se define el peso del camino C como la suma de los pesos de las aristas que lo componen:

$$w(C) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Definición (Distancia entre vértices)

Para dos vértices distintos $u, v \in V$, la distancia $d_G(u, v)$ es:

$$d_G(u, v) = \text{mín}\{w(C) \mid C \text{ es un } u\text{-}v \text{ camino}\}$$

Si no hay camino, $d_G(u, v) = +\infty$.

Principio de optimalidad y Relajación

Principio de optimalidad de Bellman
Si un camino es mínimo, cualquier subcamino contenido en él también es un camino mínimo entre sus propios extremos.

Demostración.

Si existiera un subcamino más corto, el peso total del camino original podría reducirse, lo que contradice que era el mínimo. □

Operación de relajación
El algoritmo mantiene para cada vértice v una estimación de su distancia mínima $d[v]$ desde la fuente s (inicialmente $d[s] = 0$ y $d[v] = +\infty$ para el resto).
Para cada arista $(u, v) \in E$:

$$\text{si } d[u] + w(u, v) < d[v] \implies d[v] \leftarrow d[u] + w(u, v)$$

UNIVERSIDAD DE CÓRDOBA

Condiciones y Complejidad

Condición de pesos no negativos

Requiere que la función de peso sea $w : E \to \mathbb{R}^+$. Pesos negativos invalidarían la estrategia *greedy*, ya que una distancia "finalizada" podría reducirse posteriormente por un arco negativo.

Complejidad computacional

Estructura de datos	**Complejidad temporal**
Lista o Arreglo simple	$O(\|V\|^2)$
Heap binario	$O((\|V\| + \|E\|) \log \|V\|)$
Heap de Fibonacci	$O(\|E\| + \|V\| \log \|V\|)$

UNIVERSIDAD DE CÓRDOBA

Ejemplo: Desarrollo paso a paso

Pseudocódigo del algoritmo

```
function Dijkstra(Graph, source):
   dist[source] = 0
   for each vertex v in Graph:
       if v != source:
           dist[v] = infinity
       add v to Q (unvisited set)
   while Q is not empty:
       u = vertex in Q with min dist[u]
       remove u from Q
       for each neighbor v of u in Q:
           alt = dist[u] + length(u, v)
           if alt < dist[v]:
               dist[v] = alt
               prev[v] = u
   return dist, prev
```

UNIVERSIDAD DE CÓRDOBA

Simulación del algoritmo

En el siguiente enlace encontrarás una simulación **desarrollada por nosotros** del algoritmo programa en `react.js`:

https://djkistra.vercel.app/

UNIVERSIDAD DE CÓRDOBA

Conclusiones

Bibliografía y webgrafía

- Insertar fuentes.